Vg Analysis

Reed A. Cartwright

Vizualizing Full Haplotypes

We have created a VCF of the Vg region based on 543 diploid samples, and for this analysis we will investigate the biallelic sites with known ancestral allele.

```
raw_vcf <- read.vcfR(here("results", "apis_mellifera_vg.dna.vcf"),
                     verbose = FALSE)
raw_vcf_fix <- vcfR2tidy(raw_vcf, info_only = TRUE)$fix

vcf <- raw_vcf[is.polymorphic(raw_vcf) &
               is.biallelic(raw_vcf) &
               raw_vcf_fix$AA != "."]
vcf_fix <- vcfR2tidy(vcf, info_only = TRUE)$fix
```

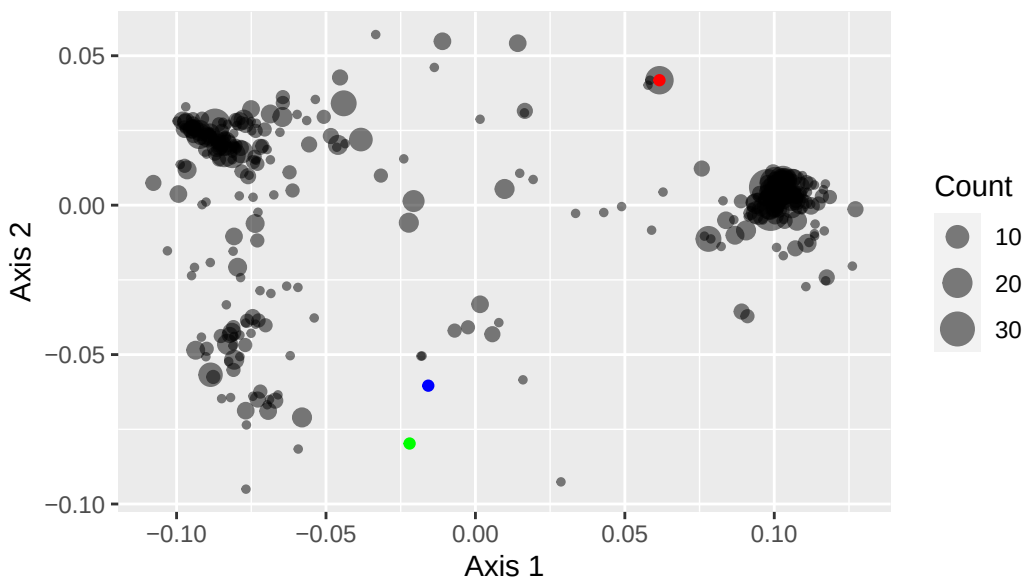
Next we will extract phased haplotypes and encode variants as 0 = ancestral and 1 = derived. We will call these our full haplotypes.

```
full_haplotypes_mat <- (extract.haps(vcf, verbose = FALSE) != vcf_fix$AA)*1L
colnames(full_haplotypes_mat) <- colnames(full_haplotypes_mat) |>
  str_replace_all(c("_0$" = "_HP1", "_1$" = "_HP2"))
full_haplotypes_str <- apply(full_haplotypes_mat, 2, str_flatten)
full_reference <- str_flatten((vcf_fix$REF != vcf_fix$AA)*1L)
full_ancestor <- str_dup("0", nrow(vcf_fix))

full_cerana <- str_flatten(case_when(
  vcf_fix$Cerana == vcf_fix$AA ~ "0",
  vcf_fix$Cerana == vcf_fix$REF ~ "1",
  vcf_fix$Cerana == vcf_fix$ALT ~ "1",
  TRUE ~ "2"
))
```

Next we will count how abundant each haplotype is in our dataset and plot the haplotypes in two dimensions using principle-coordinate analysis (PCoA).

```
haplotypes_tab <- tibble(full = full_haplotypes_str) |> count(full)
full_dist <- stringdistmatrix(c(full_reference, full_ancestor,
                                full_cerana,
                                haplotypes_tab$full),
                                method = "hamming")
v <- cmdscale(full_dist/nrow(haplotypes_tab), 2)
haplotypes_tab$x <- v[-c(1:3),1]
haplotypes_tab$y <- v[-c(1:3),2]
```

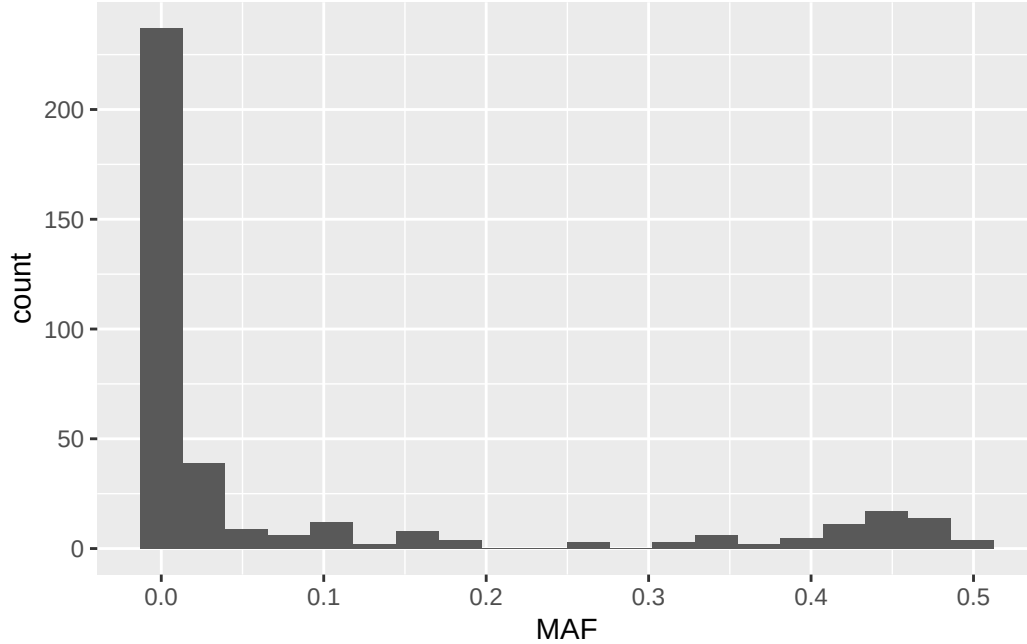
```
full_reference_xy <- list(x = v[1,1], y = v[1,2])
full_ancestor_xy <- list(x = v[2,1], y = v[2,2])
full_cerana_xy <- list(x = v[3,1], y = v[3,2])

gg <- ggplot(haplotypes_tab, aes(x = x, y = y, size = n)) +
  geom_point(alpha = 0.5)
gg <- gg + annotate("point", color = "red",
                    x = full_reference_xy$x,
                    y = full_reference_xy$y)
gg <- gg + annotate("point", color = "blue",
                    x = full_ancestor_xy$x,
                    y = full_ancestor_xy$y)
gg <- gg + annotate("point", color = "green",
                    x = full_cerana_xy$x,
                    y = full_cerana_xy$y)
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + coord_fixed()
gg
```



Vizualizing Base Haplotypes

The spread along Axis 1 reflects the presence of two genetic backgrounds that are present in our dataset at high frequency. When we look at a histogram of MAF values, we see there are 65 sites with MAF > 0.25.

```
gg <- ggplot(vcf_fix, aes(x = MAF)) + geom_histogram(bins = 20)
gg
```



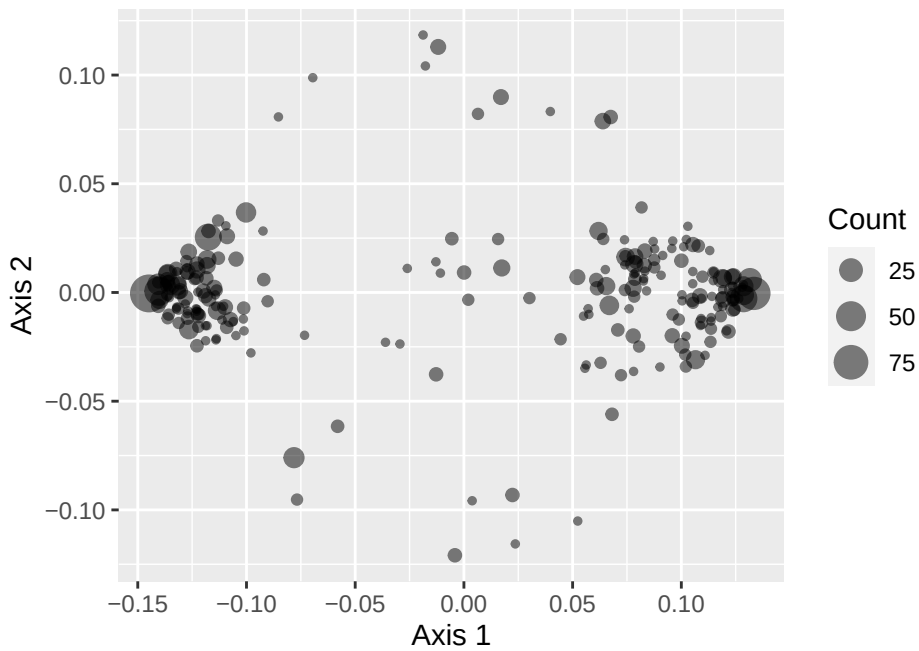
Looking only at sites with MAF > 0.25, we convert our full haplotypes into base haplotypes. Importantly, the two most common patterns do not share any derived alleles. This suggests that each of these base haplotypes represents a haplogroup that evolved independent and in an isolated population. We will use these two haplotypes as reference haplotypes for each haplogroup.

```
# select sites with high MAF
base_o <- vcf_fix$MAF > 0.25

# create base haplotypes
base_haplotypes_mat <- (extract.haps(vcf[base_o], verbose = FALSE) !=
                    vcf_fix[base_o, ]$AA)*1L
colnames(base_haplotypes_mat) <- colnames(base_haplotypes_mat) |>
  str_replace_all(c("_0$" = "_HP1", "_1$" = "_HP2"))
base_haplotypes_str <- apply(base_haplotypes_mat, 2, str_flatten)

# Count base sequences
base_tab <- tibble(base = base_haplotypes_str) |> count(base)
base_tab |> slice_max(n, n = 6) |> knitr::kable()
```

base	n
00100100000111100000010110011011111101101110010111111100011101111	95
11011011111000011111101001100100000010010001101000000011100010000	67
00100100000111100000010110011011111101001110010111111100011101111	46
11011011111000011111101001100100000000010001101000000011100010000	39
00100100000111100000010110011011111101101110010110010000011100110	37
11011011111000011111101001100100000010010001101000000011100010000	25

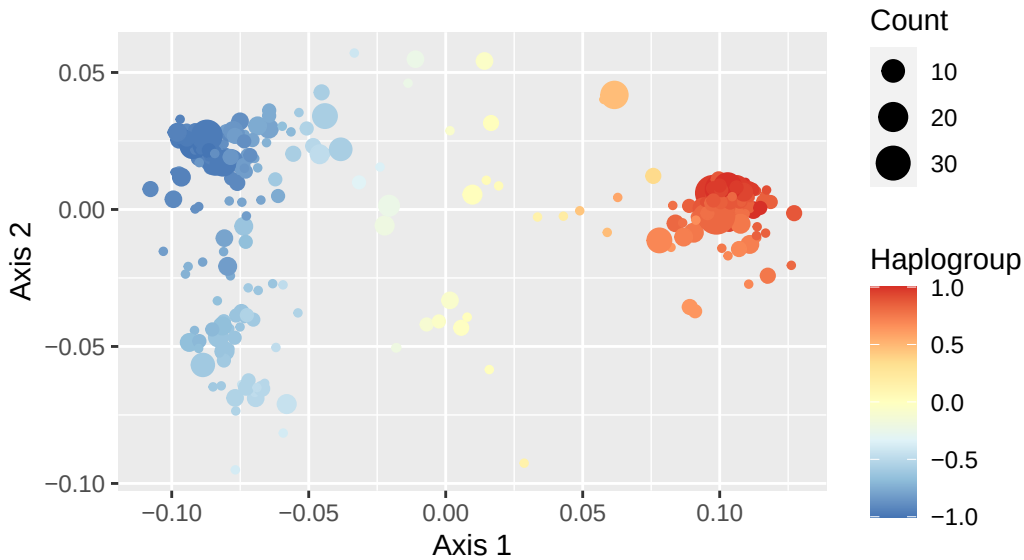
If we plot the base haplotypes in two dimensions we see that the two references separate on different ends of Axis 1.

```
base_dist <- stringdistmatrix(base_tab$base, method = "hamming")
v <- cmdscale(base_dist/nrow(base_tab), 2)
base_tab$x <- v[,1]
base_tab$y <- v[,2]

gg <- ggplot(base_tab, aes(x = x, y = y, size = n)) + geom_point(alpha = 0.5)
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + coord_fixed()
gg
```



Now we will redraw the original plot, but colored according to haplogroup.

```
base_ref <- base_tab |> slice_max(n, n = 1) |> pull(base)

haplotypes_tab$base <- haplotypes_tab$full |>
  str_sub_all(which(base_o), which(base_o)) |>
  map_chr(str_flatten)

haplotypes_tab$base_dist <- stringdistmatrix(
  haplotypes_tab$base, base_ref, "hamming")[,1]/sum(base_o)

gg <- ggplot(haplotypes_tab, aes(x = x, y = y, size = n, color = -1+2*(1-base_dist))) +
  geom_point()
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + scale_color_distiller("Haplogroup", palette = "RdYlBu")
gg <- gg + coord_fixed()
gg
```

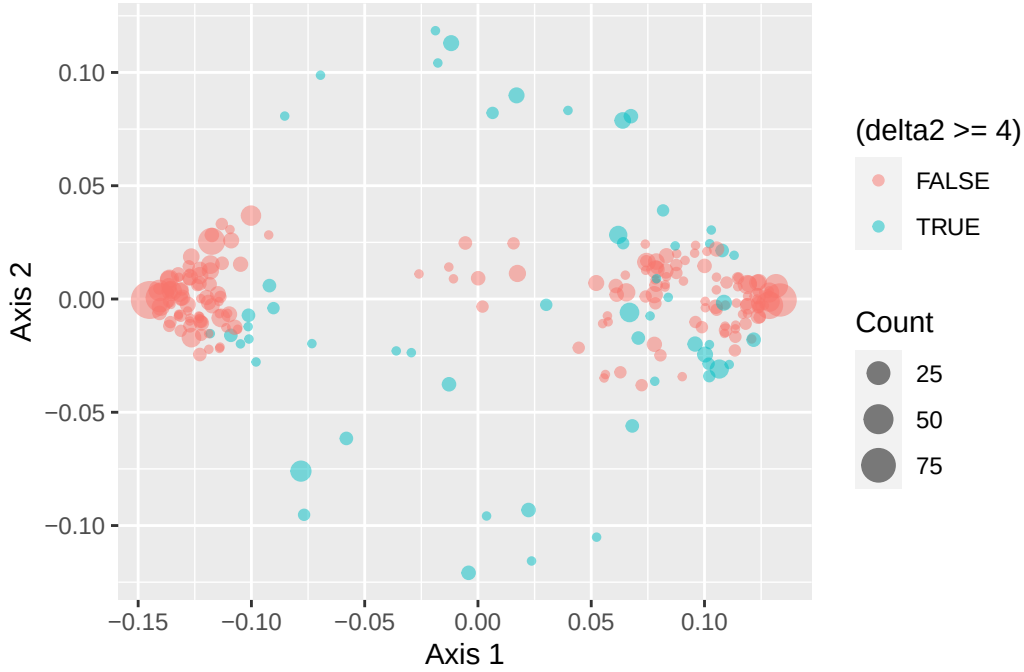


Recombinant Sequences

We can identify recombinant sequences by looking for base haplotypes that share fragments of the two base refrences. Because the base references contain a mixture of derived and ancestral alleles, we will use the $\Delta_{m,n,2}$ statistic to detect recombinant base haplotypes.

```
base_ref <- base_tab |> slice_max(n, n = 2)
base_ref_1 <- base_ref$base[1]
base_ref_2 <- base_ref$base[2]
base_anc <- str_dup("0", str_length(base_ref_1))

base_ref_1_v <- str_split_1(base_ref_1, "")
base_ref_2_v <- str_split_1(base_ref_2, "")

base_v <- str_split(base_tab$base, "") |>
  map(function(x) {
    case_when(
      x == base_ref_1_v ~ "1",
      x == base_ref_2_v ~ "2"
    )
  })
base_tab$base2 <- map_chr(base_v, str_flatten)

base_v <- str_split(base_tab$base, "") |>
  map(function(x) {
    case_when(
      x == "0" ~ "0",
      x == base_ref_1_v ~ "1",
      x == base_ref_2_v ~ "2"
    )
  })

base_tab$base3 <- map_chr(base_v, str_flatten)

# Calculate a delta statistics
```

```r
# https://mol.ax/software/delta
# https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1894573/
# https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5850291/
n <- str_length(base_ref_1)
df <- expand_grid(i = 1:(n-1))
r1 <- df |> mutate(s = str_c(
  str_sub(base_ref_1, 1, i),
  str_sub(base_ref_2, i+1, n)
))
r2 <- df |> mutate(s = str_c(
  str_sub(base_ref_2, 1, i),
  str_sub(base_ref_1, i+1, n)
))

df <- expand_grid(i = 1:(n-1), j = 1:(n-1)) |> filter(i < j)

r3 <- df |> mutate(s = str_c(
  str_sub(base_ref_1, 1, i),
  str_sub(base_ref_2, i+1, j),
  str_sub(base_ref_1, j+1, n)
  ))
r4 <- df |> mutate(s = str_c(
  str_sub(base_ref_2, 1, i),
  str_sub(base_ref_1, i+1, j),
  str_sub(base_ref_2, j+1, n)
  ))

rec_set <- c(base_anc, base_ref_1, base_ref_2, r1$s, r2$s, r3$s, r4$s)

d <- stringdistmatrix(base_tab$base, rec_set, method = "hamming")

base_tab$delta0 <- d[,1] - apply(d[,1:3], 1, min)
base_tab$delta1 <- apply(d[,1:3], 1, min) - apply(d[,1:(2*n+1)], 1, min)
base_tab$delta2 <- apply(d[,1:3], 1, min) - apply(d, 1, min)

gg <- ggplot(base_tab, aes(x = x, y = y, size = n, color=(delta2 >= 4))) +
  geom_point(alpha = 0.5)
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + coord_fixed()
gg
```




```
haplotypes_tab_orig <- haplotypes_tab
df <- base_tab |> select(base, base2, base3, delta0, delta1, delta2)
haplotypes_tab <- left_join(haplotypes_tab_orig, df) |>
  mutate(is_recomb = delta2 >= 4)
```

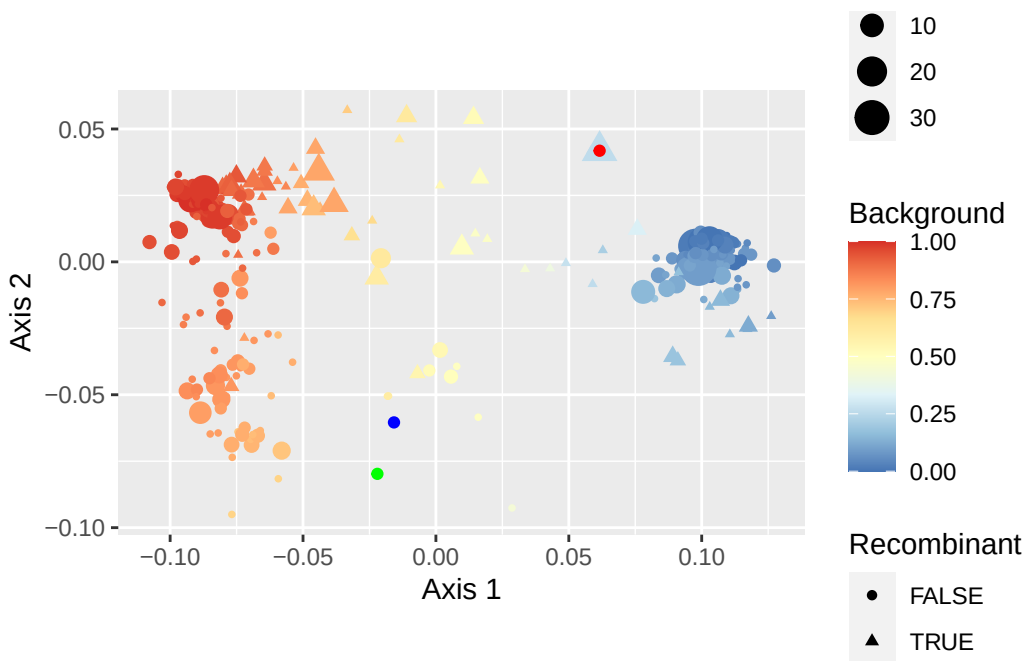
```
Joining with `by = join_by(base)`
```

```
# Write recombination information to results
full_tab <- enframe(full_haplotypes_str, name = "id", value = "full")
full_tab <- full_tab |> separate_wider_regex(id, c("[^_]+_",
    "apiary" = "..", ".*"), cols_remove = FALSE) |>
    separate_wider_delim(id, "_", names = c("sample", "name", NA),
                         cols_remove = FALSE)
full_tab <- full_tab |> mutate(region = case_match(apiary,
    c("NC", "CA", "MD", "AZ", "MN", "IL") ~ "USA",
    c("FN", "SW", "RN", "CS", "DK", "IR", "PO", "TX", "FR") ~ "Brownbee",
    c("SL", "IT", "PL", "MK", "MT", "TR") ~ "South Europe"
    ))
full_tab <- full_tab |> relocate(id, name, sample, apiary, region)
full_tab <- full_tab |> left_join(haplotypes_tab)
```

```
Joining with `by = join_by(full)`
```

```
write_csv(full_tab, here("results", "full_haplotypes.csv.gz"))

gg <- ggplot(haplotypes_tab, aes(x = x, y = y, size = n, color = base_dist,
                     shape = is_recomb)) + geom_point()
gg <- gg + annotate("point", color = "red",
                    x = full_reference_xy$x,
                    y = full_reference_xy$y)
```

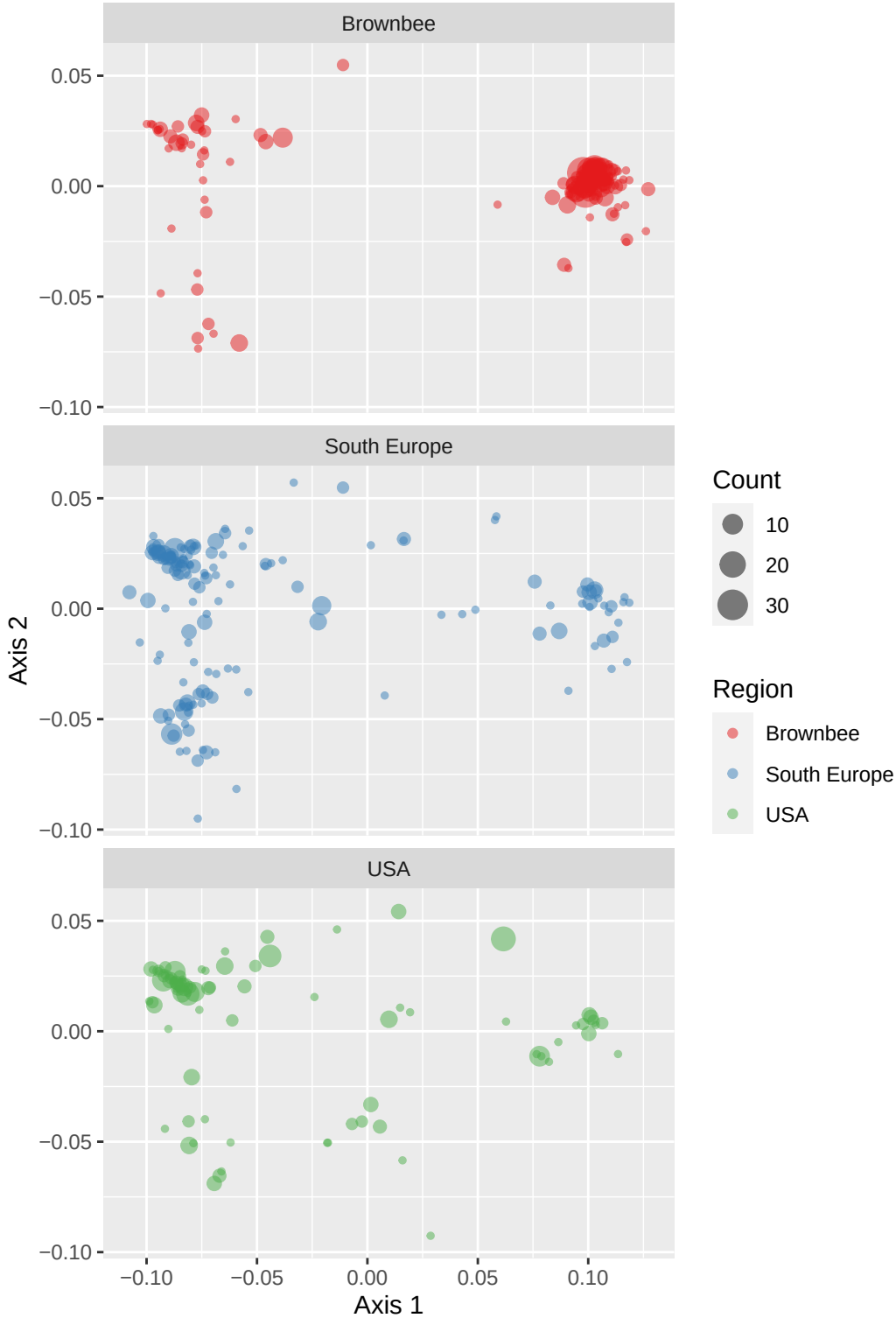
```
gg <- gg + annotate("point", color = "blue",
                     x = full_ancestor_xy$x,
                     y = full_ancestor_xy$y)
gg <- gg + annotate("point", color = "green",
                     x = full_cerana_xy$x,
                     y = full_cerana_xy$y)
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + scale_shape_discrete("Recombinant")
gg <- gg + scale_color_distiller("Background", palette = "RdYlBu")
gg <- gg + coord_fixed()
gg
```



Geographic Regions

Same analysis as before but now we will split it based on broad geographic regions

```
full_by_region <- full_tab |> count(full, region, x, y, sort = TRUE)

gg <- ggplot(full_by_region, aes(x = x, y = y, size = n, color = region)) +
  geom_point(alpha = 0.5)
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + scale_color_brewer("Region", palette = "Set1")
gg <- gg + coord_fixed()
gg <- gg + facet_wrap(vars(region), ncol = 1)
gg
```

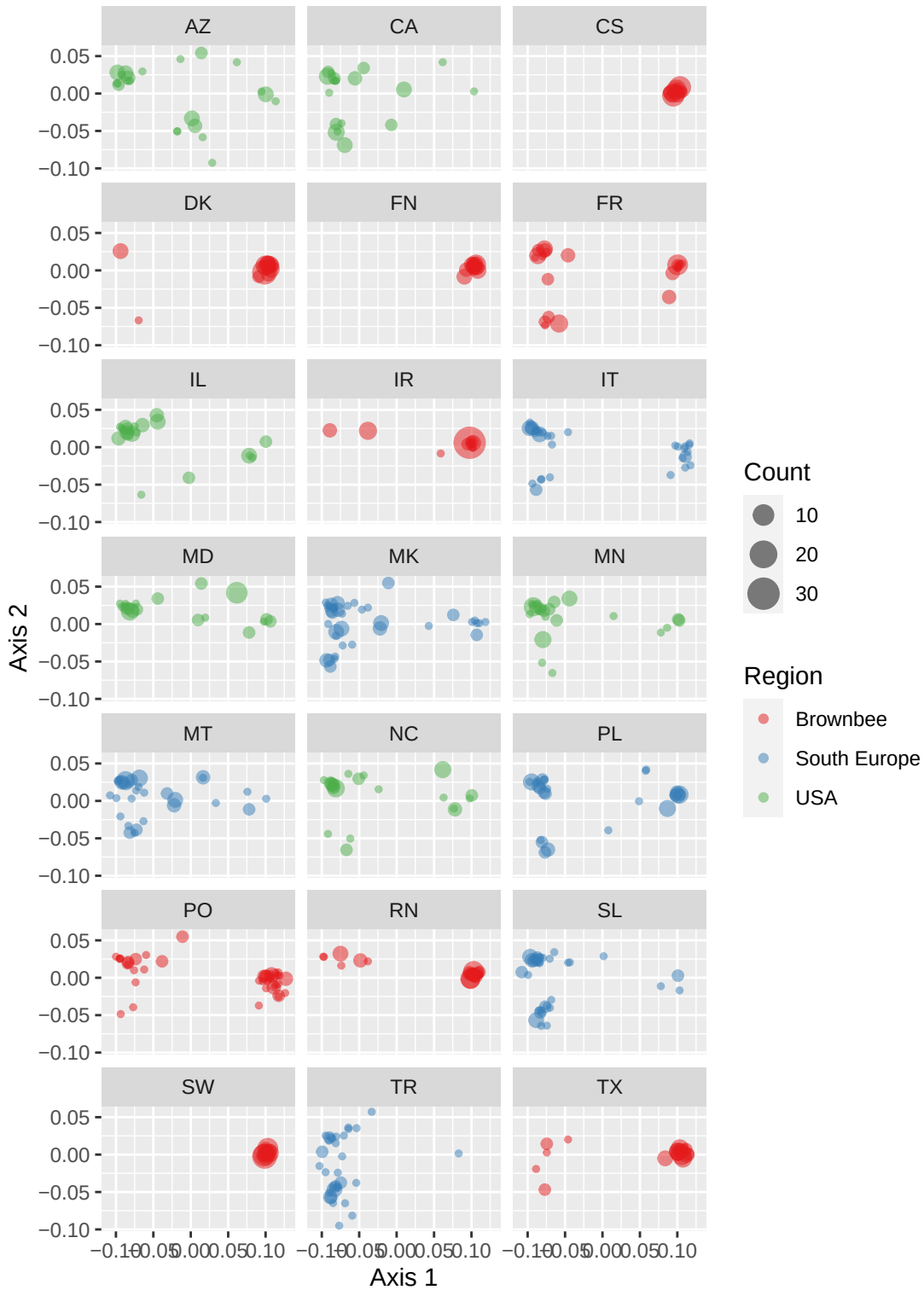


Apiary Analysis

We will also separate haplotypes by apiary and visualize them as before.

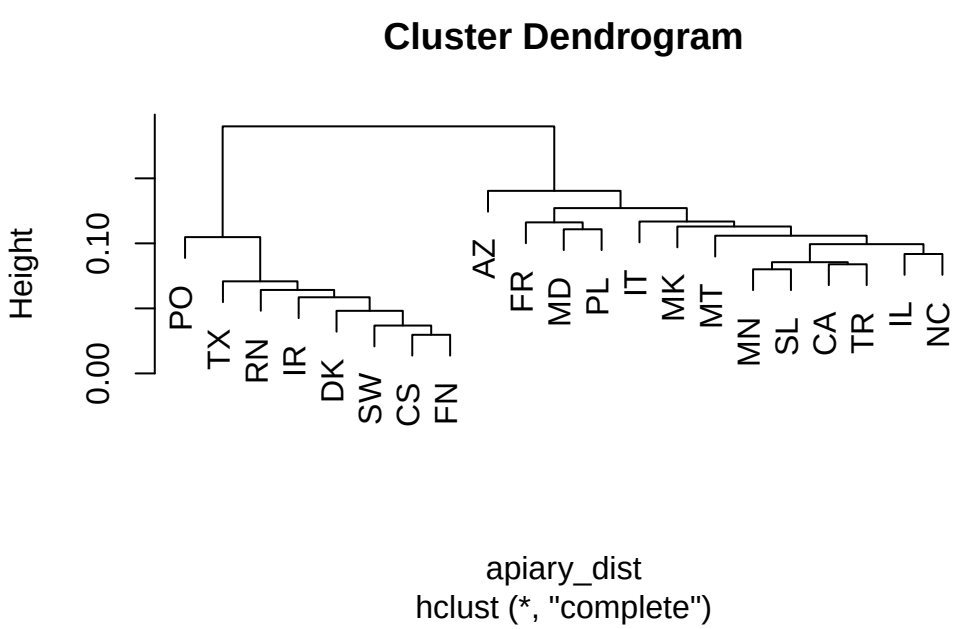
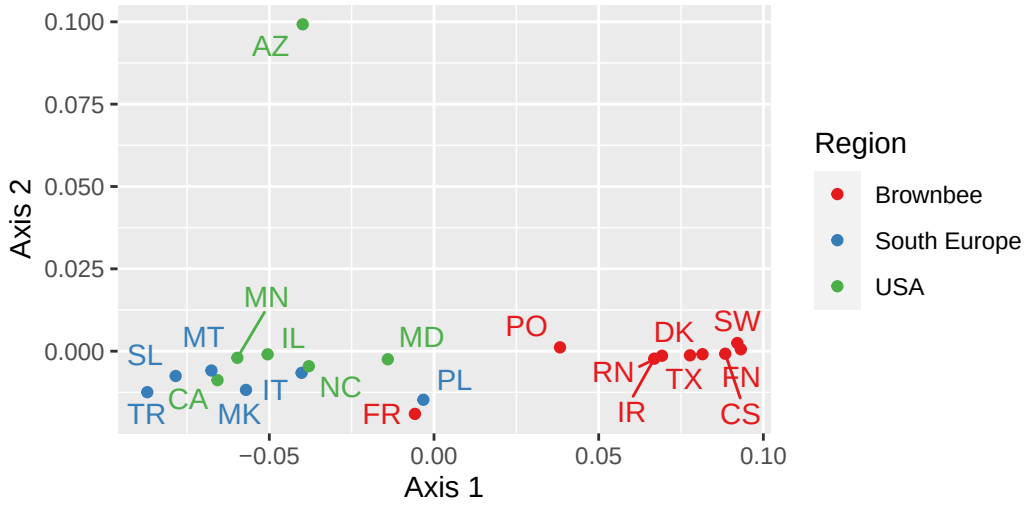
```
full_by_apiary <- full_tab |> count(full, apiary, region, x, y, sort = TRUE)

# TODO: Sort apiaries by region
gg <- ggplot(full_by_apiary, aes(x = x, y = y, size = n, color = region)) +
  geom_point(alpha = 0.5)
```

```
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_size_continuous("Count")
gg <- gg + scale_color_brewer("Region", palette = "Set1")
gg <- gg + coord_fixed()
gg <- gg + facet_wrap(vars(apiary), ncol = 3)
gg
```



We also calculate the distances between pairs of apiaries as the average distance between haplotypes taken from different apiaries.

```r
# calculate the distances between each sequence pair
tab <- full_tab |> arrange(apiary)
full_dist <- stringdistmatrix(tab$full, method = "hamming") /
  nrow(haplotypes_tab)

# construct a table to hold the pairwise distance data
x <- combn(nrow(tab), 2)
a <- x[1, ]
b <- x[2, ]
full_dist_tab <- tibble(
  id_1 = tab$id[a],
  id_2 = tab$id[b],
  apiary_1 = tab$apiary[a],
  apiary_2 = tab$apiary[b],
  dist = full_dist )

# calculate the mean distance between pairs of apiaries
apiary_dist_tab <- summarize(full_dist_tab, d = mean(dist),
                             .by = c(apiary_1, apiary_2))

# convert results into a distance object
labs <- unique(tab$apiary)
apiary_dist <- structure(filter(apiary_dist_tab, apiary_1 != apiary_2)$d,
                         Size = length(labs),
                         Labels = labs,
                         Diag = FALSE,
                         Upper = FALSE,
                         class = "dist")

# plot a hierarchical clustering of the apiaries
plot(hclust(apiary_dist))
```



```r
# plot a PCoA analysis of apiaries
v <- cmdscale(apiary_dist, 2)
apiary_xy <- tibble(apiary = rownames(v), x = v[,1], y = v[,2])
apiary_xy <- apiary_xy |> mutate(region = case_match(apiary,
    c("NC", "CA", "MD", "AZ", "MN", "IL") ~ "USA",
    c("FN", "SW", "RN", "CS", "DK", "IR", "PO", "TX", "FR") ~ "Brownbee",
    c("SL", "IT", "PL", "MK", "MT", "TR") ~ "South Europe"
    ))

gg <- ggplot(apiary_xy, aes(x = x, y = y, label = apiary, color = region)) +
  geom_point() + geom_text_repel(show.legend = FALSE)
gg <- gg + coord_fixed()
gg <- gg + xlab("Axis 1") + ylab("Axis 2")
gg <- gg + scale_color_brewer("Region", palette = "Set1")
gg
```